

Programmazione dinamica pseudo-polinomiale per il problema

$$P_m \| C_{\max}$$

1 Descrizione del problema

Consideriamo il problema di scheduling su m macchine parallele identiche, indicato nella notazione a tre campi di Graham et al. come

$$P_m \| C_{\max}.$$

Sono dati n job indipendenti $J = \{1, \dots, n\}$, ciascuno con tempo di processamento intero positivo $p_j \in \mathbb{Z}_+$. Ogni job deve essere assegnato a una e una sola macchina, senza preemption. Le macchine sono identiche, quindi un job richiede lo stesso tempo di processamento su qualunque macchina.

L'obiettivo è minimizzare il *makespan*, cioè il massimo carico tra le macchine:

$$C_{\max} = \max_{h=1, \dots, m} L_h,$$

dove L_h indica il carico totale dei job assegnati alla macchina h .

Poiché le macchine sono parallele e identiche e non vi sono release date, due date o altri vincoli temporali, l'ordine dei job su una macchina non influenza il valore della soluzione. Il problema si riduce quindi a partizionare gli n job in m sottoinsiemi, minimizzando il massimo carico dei sottoinsiemi.

Poniamo

$$P = \sum_{j=1}^n p_j.$$

Nel seguito si assume che m sia fissato. In questo caso il problema ammette un algoritmo di programmazione dinamica pseudo-polinomiale.

2 Il caso di due macchine

Consideriamo dapprima il problema

$$P_2 \| C_{\max}.$$

Con due macchine è sufficiente conoscere quali carichi possono essere ottenuti sulla prima macchina. Il carico della seconda macchina è infatti determinato implicitamente dal carico totale dei job già assegnati.

Per $i = 0, \dots, n$, definiamo

$$R_i \subseteq \{0, \dots, P\}$$

come l'insieme dei carichi ottenibili sulla prima macchina utilizzando solo i primi i job.

Lo stato iniziale è

$$R_0 = \{0\}.$$

Quando si considera il job i , con tempo di processamento p_i , da ogni carico raggiungibile $s \in R_{i-1}$ si hanno due possibilità:

- (i) assegnare il job i alla seconda macchina, lasciando invariato il carico della prima macchina;
- (ii) assegnare il job i alla prima macchina, aumentando il suo carico di p_i .

La ricorrenza è quindi

$$R_i = R_{i-1} \cup \{s + p_i : s \in R_{i-1}\}.$$

Alla fine, per ogni carico $s \in R_n$, il carico della seconda macchina è $P - s$. Il makespan associato è

$$\max\{s, P - s\}.$$

Pertanto il valore ottimo è

$$C_{\max}^* = \min_{s \in R_n} \max\{s, P - s\}.$$

Questa programmazione dinamica è sostanzialmente la stessa del problema PARTITION. Il numero di stati è al più $P + 1$ e, per ciascun job, gli stati vengono aggiornati in tempo lineare in P . La complessità è quindi

$$O(nP),$$

con memoria $O(P)$.

3 Generalizzazione a m macchine fissato

Per $m > 2$, l'idea è mantenere esplicitamente i carichi delle prime $m - 1$ macchine. Il carico della macchina m viene invece determinato implicitamente dal carico totale dei job già considerati.

Per $i = 0, \dots, n$, definiamo

$$R_i \subseteq \{0, \dots, P\}^{m-1}$$

come l'insieme dei vettori di carico raggiungibili sulle prime $m - 1$ macchine utilizzando i primi i job.

Uno stato ha quindi la forma

$$(x_1, \dots, x_{m-1}),$$

dove x_h è il carico assegnato alla macchina h , per $h = 1, \dots, m - 1$.

Lo stato iniziale è

$$R_0 = \{(0, \dots, 0)\}.$$

Supponiamo ora di avere uno stato

$$(x_1, \dots, x_{m-1}) \in R_{i-1}$$

e di considerare il job i , di durata p_i . Il job può essere assegnato a una qualunque delle m macchine.

Se il job viene assegnato a una delle prime $m - 1$ macchine, diciamo alla macchina h , si ottiene il nuovo stato

$$(x_1, \dots, x_h + p_i, \dots, x_{m-1}).$$

Se invece il job viene assegnato alla macchina m , che non è rappresentata esplicitamente nello stato, il vettore dei carichi delle prime $m - 1$ macchine resta invariato:

$$(x_1, \dots, x_{m-1}).$$

In altri termini, la transizione è data da

$$R_i = R_{i-1} \cup \bigcup_{h=1}^{m-1} \{(x_1, \dots, x_h + p_i, \dots, x_{m-1}) : (x_1, \dots, x_{m-1}) \in R_{i-1}\}.$$

Alla fine, per ogni vettore raggiungibile

$$(x_1, \dots, x_{m-1}) \in R_n,$$

il carico della macchina m è

$$x_m = P - \sum_{h=1}^{m-1} x_h.$$

Il makespan associato allo stato è quindi

$$\max \left\{ x_1, \dots, x_{m-1}, P - \sum_{h=1}^{m-1} x_h \right\}.$$

Il valore ottimo è

$$C_{\max}^* = \min_{(x_1, \dots, x_{m-1}) \in R_n} \max \left\{ x_1, \dots, x_{m-1}, P - \sum_{h=1}^{m-1} x_h \right\}.$$

4 Complessità

Il numero di vettori di carico possibili per le prime $m - 1$ macchine è al più

$$(P + 1)^{m-1}.$$

Per ciascun job e per ciascuno stato, si generano al più m transizioni. Pertanto una stima diretta della complessità è

$$O(nmP^{m-1}).$$

Se m è fissato, questa complessità è pseudo-polinomiale. Per esempio:

$$m = 2 \quad \Rightarrow \quad O(nP),$$

$$m = 3 \quad \Rightarrow \quad O(nP^2),$$

$$m = 4 \quad \Rightarrow \quad O(nP^3).$$

La memoria può essere ridotta mantenendo soltanto due livelli della programmazione dinamica, cioè R_{i-1} e R_i . In tal modo, lo spazio richiesto è

$$O(P^{m-1}).$$

5 Esempio per tre macchine

Per $m = 3$, la DP mantiene stati del tipo

$$(x_1, x_2),$$

cioè i carichi delle prime due macchine. Il carico della terza macchina è implicito.

Da uno stato (x_1, x_2) , considerando un job i di durata p_i , si generano i seguenti stati:

$$(x_1 + p_i, x_2),$$

$$(x_1, x_2 + p_i),$$

$$(x_1, x_2).$$

Il terzo caso corrisponde all'assegnazione del job alla terza macchina.

Alla fine, se il carico totale è P , lo stato (x_1, x_2) rappresenta uno schedule con carichi

$$x_1, \quad x_2, \quad P - x_1 - x_2.$$

Il makespan associato è

$$\max\{x_1, x_2, P - x_1 - x_2\}.$$

Riferimenti bibliografici

- [1] Sahni, S. K. (1976). Algorithms for scheduling independent tasks. *Journal of the ACM*, 23(1), 116–127.
- [2] Graham, R. L., Lawler, E. L., Lenstra, J. K., and Rinnooy Kan, A. H. G. (1979). Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*, 5, 287–326.